

Guia de estudio

Seccion Proveedores - Sistema Empresa

Para explicar que hace tu parte, como se relaciona con compras y por que esta organizada por capas.

Idea central	La seccion de proveedores administra a las empresas/personas a quienes la empresa les compra.
Por que importa	Sin proveedores validos no se deben registrar compras. Proveedor es la base del modulo de compras.
Decision clave	No se elimina un proveedor: se desactiva para conservar historial y evitar romper compras existentes.

1. Resumen para decirlo en 30 segundos

Mi seccion es Proveedores. Sirve para registrar, consultar, buscar y desactivar proveedores. Esta conectada con compras porque una compra necesita un idProveedor valido y activo. El codigo esta dividido en capas: el modelo guarda datos, el controller recibe solicitudes, el service valida reglas de negocio y el repository hace las consultas SQL a MySQL.

2. Arquitectura por capas

Archivo	Rol	Como lo explico
Proveedor.java	Modelo	Representa los datos del proveedor: id, nombre, RFC, telefono, correo, direccion y activo.
ProveedorController.java	Controller	Es el puente entre App y Service. Recibe la accion y la manda a la logica.
ProveedorService.java	Service	Contiene las validaciones: campos obligatorios, correo con @, RFC no repetido y existencia antes de desactivar.
ProveedorRepository.java	Repository	Ejecuta SQL: INSERT, SELECT, COUNT y UPDATE contra la tabla Proveedor.
Conexion.java	Soporte	Crea la conexion a MySQL para que los repositorios puedan operar.

3. Flujo de registrar proveedor

- App pide nombre, RFC, telefono, correo y direccion al usuario.
- ProveedorController.registrarProveedor recibe esos datos y los envia al Service.
- ProveedorService valida nombre, RFC y telefono obligatorios.
- ProveedorService valida que el correo contenga @.
- ProveedorService pregunta al Repository si el RFC ya existe.
- Si todo es correcto, crea un Proveedor con activo = true.
- ProveedorRepository.guardar usa un INSERT para registrar el proveedor en MySQL.

Ruta mental:

App -> ProveedorController -> ProveedorService -> ProveedorRepository -> MySQL

4. Validaciones que debes defender

Validacion	Donde esta	Por que existe
Nombre obligatorio	ProveedorService.validarTexto	Sin nombre no se identifica a quien se le compra.
RFC obligatorio	ProveedorService.validarTexto	El RFC identifica fiscalmente al proveedor.

Validacion	Donde esta	Por que existe
Telefono obligatorio	ProveedorService.validarTexto	Permite contacto si hay problemas con una compra.
Correo con @	ProveedorService.registrarProveedor	Evita guardar un contacto claramente invalido.
RFC no duplicado	ProveedorRepository.existeRfc	Evita tener dos registros fiscales iguales.
Existencia antes de desactivar	ProveedorService.desactivarProveedor	No se modifica un proveedor inexistente.

5. Relacion con compras

La relacion mas importante es que Compra usa idProveedor. En CompraService.registrarCompra se revisa si el proveedor existe y esta activo. Esa revision la hace CompraRepository.existeProveedor con esta condicion:

```
SELECT COUNT(*) FROM Proveedor WHERE id_proveedor = ? AND activo = ?
```

Esto significa que tu seccion controla quien puede participar en una compra. Si el proveedor esta inactivo, la compra no debe registrarse.

6. Por que se desactiva y no se elimina

- Porque pueden existir compras anteriores relacionadas con ese proveedor.
- Si se borrara, se perderia contexto historico y podrian romperse relaciones en base de datos.
- Con activo = false se bloquean nuevas compras, pero se conserva la informacion para consultas o auditoria.
- Es una decision comun en sistemas administrativos: baja logica en lugar de borrar fisicamente.

7. Metodos principales para explicar

Metodo	Explicacion corta
registrarProveedor(...)	Valida datos, evita RFC repetido, crea el objeto y lo guarda activo.
mostrarProveedores()	Devuelve la lista completa de proveedores registrados.
buscarProveedor(idProveedor)	Busca por id y lanza error si no existe.
desactivarProveedor(idProveedor)	Verifica existencia y cambia activo a false.
existeRfc(rfc)	Cuenta registros con ese RFC para impedir duplicados.
mapear(ResultSet)	Convierte una fila SQL en un objeto Proveedor.

8. Preguntas que te pueden hacer

Pregunta	Respuesta segura
Por que hay Controller, Service y Repository?	Para separar responsabilidades. El Controller recibe acciones, el Service valida reglas y el Repository maneja SQL.

Pregunta	Respuesta segura
Por que el modelo no valida?	Porque el modelo solo representa datos. Las reglas se concentran en el Service para que sea mas facil mantenerlas.
Por que se valida RFC repetido?	Porque el RFC identifica al proveedor fiscalmente y duplicarlo causaria datos confusos en compras.
Por que correo solo revisa @?	Es una validacion basica adecuada para el alcance del proyecto. Si el sistema creciera, podria hacerse una validacion mas estricta.
Que pasa si intento comprar a un proveedor inactivo?	CompraService lo rechaza porque CompraRepository.existeProveedor solo acepta proveedores con activo = true.
Que parte toca la base de datos?	ProveedorRepository, usando Conexion.getConnection y PreparedStatement.

9. Guion de exposicion

- Primero diria que mi modulo administra proveedores, que son necesarios para registrar compras.
- Despues explicaria la division por capas: Proveedor, Controller, Service y Repository.
- Luego mostraria el flujo de registrar proveedor, desde App hasta MySQL.
- Despues defenderia las validaciones: campos obligatorios, correo, RFC unico y desactivacion segura.
- Por ultimo explicaria la relacion con compras: si el proveedor no existe o esta inactivo, no se puede crear la compra.

10. Frases utiles para defender decisiones

- La responsabilidad de Proveedor.java es representar datos, no decidir reglas ni guardar en SQL.
- Las reglas estan en el Service para mantener la logica en un solo lugar.
- El Repository concentra SQL para que el resto del codigo no dependa de consultas.
- La desactivacion protege el historial porque no borra proveedores que pudieron tener compras.
- La relacion con Compra se basa en idProveedor y en el estado activo del proveedor.

Mini mapa final: Proveedor registra y controla disponibilidad; Compra usa proveedores activos; Repository guarda y consulta; Service protege reglas de negocio.